

Sri Lanka Institute of Information Technology



Faculty of Computing

IT2011 - Artificial Intelligence and Machine Learning

Lab 05 - Introduction to Generative AI

Submission: Exercises 2 to 6

IT25101663 - Faleel M. H.

Exercise 4: OpenAI API in Python

Baseline test (physics assistant)

```
openai.api_key = os.environ['OPENAI_API_KEY']

In [17]: # Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a helpful assistant with deep knowledge in physics."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Explain how black holes are formed."}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

Black holes are formed when a massive star runs out of fuel and collapses in on itself during a supernova explosion. As the star collapses, the outer layers are blasted into space while the core collapses under its own gravity. If the core has a mass greater than about three times that of the Sun, it will collapse into a black hole.

The core of the star becomes so dense that its gravity becomes incredibly strong, causing it to collapse to a point of infinite density called a singularity. This singularity is surrounded by an event horizon, which is the point of no return for anything that crosses it. Once an object or light crosses the event horizon, it can never escape the gravitational pull of the black hole.

Black holes come in different sizes, with stellar-mass black holes being formed from the collapse of massive stars, and supermassive black holes found at the centers of galaxies, with masses millions or even billions of times that of the Sun.

Event Planner Chatbot

System: "You are an expert event planner..."

User: "I'm planning a birthday party for my friend turning 21, 20 guests, \$200 budget, at home.
Give a full plan."

```
In [19]: # Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are an expert event planner with 15 years of experience organizing parties."}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "I'm planning a birthday party for my friend who is turning 21. We're expecting 20 guests, with a budget of $200, at home. Can you give me a full plan?"}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

Absolutely! Here's a general plan for your friend's 21st birthday party:

Venue Setup:

- Create a designated party area in your living room or backyard with decorations like balloons, banners, and fairy lights to set a festive mood.
- Set up a photo booth corner with props for guests to take fun pictures.
- Arrange seating for guests to mingle and enjoy the party.
- Have a designated area for food and drinks.

Food:

- Opt for a budget-friendly option like a DIY taco bar or a pasta station where guests can customize their meals.
- Consider making or ordering some finger foods like sliders, mini pizzas, and chips with dips.
- Don't forget a birthday cake or cupcakes to celebrate the occasion.

Activities:

- Plan party games like beer pong, flip cup, or a themed quiz related to the birthday person.
- Prepare a playlist of your friend's favorite songs or a mix of popular party hits for background music.
- Have a karaoke setup for guests to sing their hearts out.

Rough Timeline:

- 2:00 PM: Set up the party area and decorate the space.
- 3:00 PM: Guests start arriving, greet them, and offer them drinks.
- 3:30 PM: Begin the activities and games to get everyone involved.
- 5:00 PM: Serve the main food and have the birthday cake cutting.
- 6:00 PM: Continue with activities and let guests enjoy the party.
- 8:00 PM: Thank guests for attending and take group photos.

Remember to personalize the party based on your friend's preferences and interests. Have a backup plan for outdoor activities in case of unexpected weather changes. Enjoy planning and hosting the party!

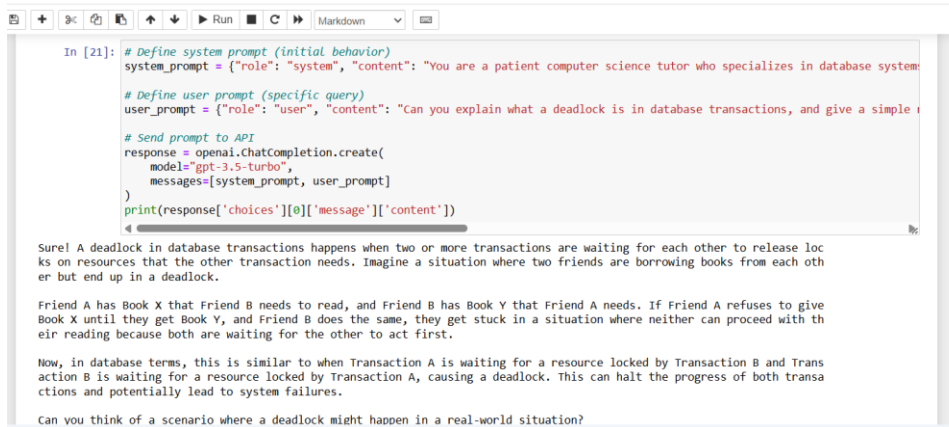
helper function

This may look familiar if you took the earlier course "ChatGPT Prompt Engineering for Developers" Course.

Unique Prompt: Database Systems Tutor

System: "You are a patient CS tutor specializing in database systems..."

User: "Explain what a deadlock is in database transactions, with a real-world analogy."



```
In [21]: # Define system prompt (initial behavior)
system_prompt = {"role": "system", "content": "You are a patient computer science tutor who specializes in database systems"}

# Define user prompt (specific query)
user_prompt = {"role": "user", "content": "Can you explain what a deadlock is in database transactions, and give a simple real-world analogy?"}

# Send prompt to API
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[system_prompt, user_prompt]
)
print(response['choices'][0]['message']['content'])
```

Sure! A deadlock in database transactions happens when two or more transactions are waiting for each other to release locks on resources that the other transaction needs. Imagine a situation where two friends are borrowing books from each other but end up in a deadlock.

Friend A has Book X that Friend B needs to read, and Friend B has Book Y that Friend A needs. If Friend A refuses to give Book X until they get Book Y, and Friend B does the same, they get stuck in a situation where neither can proceed with their reading because both are waiting for the other to act first.

Now, in database terms, this is similar to when Transaction A is waiting for a resource locked by Transaction B and Transaction B is waiting for a resource locked by Transaction A, causing a deadlock. This can halt the progress of both transactions and potentially lead to system failures.

Can you think of a scenario where a deadlock might happen in a real-world situation?

Changing the system prompt clearly changed the model's persona and output style across all three runs, while the underlying API call stayed identical.